

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

SKILLING SESSION 3 – SQL: ADVANCED QUERYING

Name	Chakrika Panakala	Roll No.	2520090042
Course	25CS1302E	CO Mapping	CO1
Session	Skilling 3	Topic	SQL – Advanced Querying

Aim

To write DML statements and parameterised queries that prevent SQL injection, and to write 12+ queries for the Task Hub dashboard using all join types, subqueries, CTEs (including recursive), aggregate functions with GROUP BY/HAVING, window functions (ROW_NUMBER, RANK, LAG, LEAD), CASE expressions, JSONB operators and set operations.

1. Database and Table Creation

```
DROP DATABASE IF EXISTS taskhub_db;
CREATE DATABASE taskhub_db;
\c taskhub_db

CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    team VARCHAR(20) NOT NULL,
    manager_id INT REFERENCES users(user_id)
);

CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    owner_id INT REFERENCES users(user_id)
);

CREATE TABLE tasks (
    task_id SERIAL PRIMARY KEY,
    project_id INT REFERENCES projects(project_id),
    assignee_id INT REFERENCES users(user_id),
    title VARCHAR(200) NOT NULL,
    status VARCHAR(12) NOT NULL CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE')),
    hours_logged NUMERIC(5,1) NOT NULL DEFAULT 0,
    due_date DATE,
    metadata JSONB
);
```

2. DML Operations

Sample users, projects and tasks were inserted into the database. The task data includes status, hours logged, due dates and JSONB metadata.

```
UPDATE tasks
SET status = 'IN_PROGRESS', hours_logged = 2.0
WHERE title = 'Add JWT authentication';

DELETE FROM tasks WHERE title = 'Dark mode!';

SELECT task_id, title, status, hours_logged
FROM tasks ORDER BY task_id;
```

Output after DML

task_id	title	status	hours_logged
1	Design ER diagram	DONE	6.0
2	Create PostgreSQL schema	DONE	8.5
3	Write FastAPI CRUD	IN_PROGRESS	12.0
4	Add JWT authentication	IN_PROGRESS	2.0
5	Build dashboard page	IN_PROGRESS	9.0
6	Replace mock data	TODO	1.5
8	Set up CI pipeline	TODO	0.0

3. Parameterised Query and SQL Injection Protection

```
PREPARE tasks_by_email(VARCHAR) AS
SELECT t.title, t.status
FROM tasks t JOIN users u ON u.user_id = t.assignee_id
WHERE u.email = $1;

EXECUTE tasks_by_email('rahul@taskhub.dev');
```

```
EXECUTE tasks_by_email('' OR ''1''=''1');  
DEALLOCATE tasks_by_email;
```

For the valid email, Rahul's two tasks were returned. The injection string returned zero rows because the parameter was treated as a data value rather than executable SQL.

4. Advanced Dashboard Queries

Q1 – INNER JOIN

```
SELECT t.title, u.full_name AS assignee, p.name AS project
FROM tasks t
INNER JOIN users u ON u.user_id = t.assignee_id
INNER JOIN projects p ON p.project_id = t.project_id
ORDER BY p.name, t.title;
```

Output:

```
Add JWT authentication | Rahul Verma | Task Hub Backend
Create PostgreSQL schema | Rahul Verma | Task Hub Backend
Design ER diagram | Habeeb | Task Hub Backend
Write FastAPI CRUD | Arjun Nair | Task Hub Backend
Build dashboard page | Ananya Rao | Task Hub Frontend
Replace mock data | Sneha Reddy | Task Hub Frontend
```

Q2 – LEFT JOIN

```
SELECT u.full_name, COUNT(t.task_id) AS task_count
FROM users u LEFT JOIN tasks t ON t.assignee_id = u.user_id
GROUP BY u.full_name
ORDER BY task_count DESC, u.full_name;
```

Output:

```
Rahul Verma    2
Ananya Rao     1
Arjun Nair     1
Habeeb        1
Sneha Reddy   1
Meera Iyer     0
```

Q3 – RIGHT JOIN

```
SELECT p.name AS project, t.title
FROM tasks t RIGHT JOIN projects p ON p.project_id = t.project_id
WHERE t.task_id IS NULL OR t.status = 'TODO'
ORDER BY p.name;
```

Output:

```
Mobile App      [no task]
Task Hub Backend Set up CI pipeline
Task Hub Frontend Replace mock data
```

Q4 – FULL OUTER JOIN

```
SELECT u.full_name, t.title
FROM users u FULL OUTER JOIN tasks t ON t.assignee_id = u.user_id
WHERE u.user_id IS NULL OR t.task_id IS NULL;
```

Output:

```
[no user] Set up CI pipeline
Meera Iyer [no task]
```

Q5 – CROSS JOIN

```
SELECT s.status, tm.team
FROM (VALUES ('TODO'), ('IN_PROGRESS'), ('DONE')) AS s(status)
CROSS JOIN (SELECT DISTINCT team FROM users) AS tm
ORDER BY tm.team, s.status
LIMIT 6;
```

Output:

```
DONE      Backend
IN_PROGRESS Backend
TODO      Backend
DONE      Frontend
IN_PROGRESS Frontend
TODO      Frontend
```

Q6 – SELF JOIN

```
SELECT e.full_name AS employee,
       COALESCE(m.full_name, '(no manager)') AS manager
FROM users e LEFT JOIN users m ON m.user_id = e.manager_id
ORDER BY e.user_id;
```

Output:

```
Habeeb      (no manager)
Ananya Rao  Habeeb
Rahul Verma Habeeb
Sneha Reddy Ananya Rao
Arjun Nair  Rahul Verma
Meera Iyer  Habeeb
```

Q7 – Scalar Subquery

```
SELECT title, hours_logged
FROM tasks
```

```
WHERE hours_logged > (SELECT AVG(hours_logged) FROM tasks)
ORDER BY hours_logged DESC;
```

Output:

```
Write FastAPI CRUD      12.0
Build dashboard page    9.0
Create PostgreSQL schema 8.5
Design ER diagram      6.0
```

Q8 – Correlated Subquery with EXISTS

```
SELECT full_name
FROM users u
WHERE EXISTS (
    SELECT 1 FROM tasks t
    WHERE t.assignee_id = u.user_id AND t.status <> 'DONE'
)
ORDER BY full_name;
```

Output:

```
Ananya Rao
Arjun Nair
Rahul Verma
Sneha Reddy
```

Q9 – CTE

```
WITH stats AS (
    SELECT project_id,
           COUNT(*) AS total,
           COUNT(*) FILTER (WHERE status = 'DONE') AS done
    FROM tasks
    GROUP BY project_id
)
SELECT p.name, s.total, s.done,
       ROUND(100.0 * s.done / s.total, 1) AS percent_complete
FROM stats s JOIN projects p ON p.project_id = s.project_id
ORDER BY percent_complete DESC;
```

Output:

```
Task Hub Backend  5   2  40.0
Task Hub Frontend 2   0   0.0
```

Q10 – Recursive CTE

```
WITH RECURSIVE hierarchy AS (
    SELECT user_id, full_name, 0 AS level
    FROM users WHERE manager_id IS NULL
    UNION ALL
    SELECT u.user_id, u.full_name, h.level + 1
    FROM users u JOIN hierarchy h ON u.manager_id = h.user_id
)
SELECT REPEAT(' ', level) || full_name AS org_chart, level
FROM hierarchy
ORDER BY level, full_name;
```

Output:

```
Habeeb      0
Ananya Rao   1
Meera Iyer   1
Rahul Verma  1
Arjun Nair   2
Sneha Reddy  2
```

Q11 – GROUP BY / HAVING

```
SELECT u.team, COUNT(t.task_id) AS tasks,
       SUM(t.hours_logged) AS total_hours,
       ROUND(AVG(t.hours_logged), 2) AS avg_hours
FROM tasks t JOIN users u ON u.user_id = t.assignee_id
GROUP BY u.team
HAVING SUM(t.hours_logged) > 10
ORDER BY total_hours DESC;
```

Output:

```
Backend  4   28.5  7.13
Frontend 2   10.5  5.25
```

Q12 – ROW_NUMBER and RANK

```
SELECT p.name AS project, t.title, t.hours_logged,
       ROW_NUMBER() OVER (
           PARTITION BY t.project_id ORDER BY t.hours_logged DESC
       ) AS row_num,
       RANK() OVER (
           PARTITION BY t.project_id ORDER BY t.hours_logged DESC
       ) AS rnk
FROM tasks t JOIN projects p ON p.project_id = t.project_id
ORDER BY p.name, row_num;
```

Output:

```
Backend: Write FastAPI CRUD 12.0 1 1
Backend: Create PostgreSQL schema 8.5 2 2
Backend: Design ER diagram 6.0 3 3
Backend: Add JWT authentication 2.0 4 4
Backend: Set up CI pipeline 0.0 5 5
Frontend: Build dashboard page 9.0 1 1
Frontend: Replace mock data 1.5 2 2
```

Q13 – LAG and LEAD

```
SELECT title, due_date,
       LAG(due_date) OVER (ORDER BY due_date) AS previous_due,
       LEAD(due_date) OVER (ORDER BY due_date) AS next_due,
       due_date - LAG(due_date) OVER (ORDER BY due_date) AS gap_days
FROM tasks
ORDER BY due_date;
```

Output:

Design ER diagram	2026-09-20	–	2026-09-25
Create PostgreSQL schema	2026-09-25	2026-09-20	5
Build dashboard page	2026-10-05	2026-09-25	10
Write FastAPI CRUD	2026-10-08	2026-10-05	3
Replace mock data	2026-10-12	2026-10-08	4
Add JWT authentication	2026-10-15	2026-10-12	3
Set up CI pipeline	2026-10-20	2026-10-15	5

Q14 – CASE + JSONB ->>

```
SELECT title,
       metadata ->> 'priority' AS priority,
       CASE
         WHEN status = 'DONE' THEN 'Completed'
         WHEN due_date < DATE '2026-10-10' THEN 'Due soon'
         ELSE 'On track'
       END AS badge
FROM tasks
ORDER BY due_date;
```

Output:

Design ER diagram	high	Completed
Create PostgreSQL schema	high	Completed
Build dashboard page	medium	Due soon
Write FastAPI CRUD	medium	Due soon
Replace mock data	low	On track
Add JWT authentication	high	On track
Set up CI pipeline	medium	On track

Q15 – JSONB containment

```
SELECT title, metadata -> 'labels' AS labels
FROM tasks
WHERE metadata -> 'labels' ? 'api';
```

Output:

Write FastAPI CRUD	["api"]
Replace mock data	["ui", "api"]
Add JWT authentication	["api", "security"]

Q16 – UNION

```
SELECT full_name FROM users u JOIN projects p ON p.owner_id = u.user_id
UNION
SELECT full_name FROM users u JOIN tasks t ON t.assignee_id = u.user_id
ORDER BY full_name;
```

Output:

Ananya Rao
Arjun Nair
Habeeb
Rahul Verma
Sneha Reddy

Q17 – INTERSECT

```
SELECT owner_id AS user_id FROM projects WHERE owner_id IS NOT NULL
INTERSECT
SELECT assignee_id FROM tasks
ORDER BY user_id;
```

Output:

1
2

Q18 – EXCEPT

```
SELECT user_id FROM users
EXCEPT
SELECT assignee_id FROM tasks WHERE assignee_id IS NOT NULL;
```

Output:

6

Result

Eighteen dashboard queries (Q1–Q18) were executed successfully on PostgreSQL. The prepared statement returned Rahul's tasks for a valid email and zero rows for the injection string, showing that parameters are treated as data and not as SQL. INNER, LEFT, RIGHT, FULL, CROSS and SELF joins, scalar and correlated subqueries, CTEs, window functions, CASE, JSONB (->, ?) and UNION/INTERSECT/EXCEPT all produced the expected listings and summaries.